AIAA SciTech Forum
5-9 January 2015, Kissimmee, Florida
AIAA Guidance, Navigation, and Control Conference

10.2514/6.2015-1990

Check for updates

Reinforcement Learning Applied to a Quadrotor Guidance Law in Autonomous Flight

Jaime Junell*, Erik-Jan van Kampen†, Coen de Visser† and Qiping Chu‡
Delft University of Technology. Delft, The Netherlands

Autonomous flight of Unmanned Aerial Vehicles (UAVs) in unknown or uncertain environments can benefit from control methods that are able to learn and adapt to these conditions. This paper presents the setup and results of a high level reinforcement learning problem for both simulation and real flight tests. The problem provided is that of a quadrotor taking pictures of a disaster site. The environment is completely unknown at first and the agent must learn where the sites of interest are and the most efficient way to get there. The results show that the quadrotor agent can learn a converged, near optimal value function after many iterations. However, a non-converged value function can result in the same desirable actions with much fewer iterations. Furthermore in this paper, a research and test laboratory for ground robots and aerial vehicles is presented.

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

Nomenclature

π	policy
V^{π}	Value function of policy, π
V^{π^*}	Optimal value function of policy, π
V^*	Optimal value function of all policies
r	reward
γ	discount factor
α	Step size
ϵ	percentage in ϵ-greedy policy that greedy action will be chosen
N_a	number of possible actions

Subscript

s	current state
s'	next state

I. Introduction

Autonomous flight of Unmanned Aerial Vehicles(UAVs) and Micro Aerial Vehicles (MAVs) is challenging in the realm of control for many reasons. Dynamics of the system can be difficult to model, the vehicles are often more susceptible to unforseen disturbances, and there are on-board sensor and actuator limitations. One method to address these problems is to create learning or adaptive controllers. Reinforcement learning (RL), for example, can make a system learn the right actions to take with little or no a priori knowledge of its dynamics or environment and can adapt to changing conditions.[1] For these reasons, RL has become a promising tool for improving autonomous flight in many different types of UAVs and MAVs. This paper will focus on learning an optimal mission path using Reinforcement Learning for a Parrot® AR-drone 2

*PhD Candidate. TU Delft Aerospace Engineering, Control and Simulation. Delft, The Netherlands. AIAA Member Grade.
†Assistant Professor. TU Delft Aerospace Engineering, Control and Simulation. Delft, The Netherlands. AIAA Member Grade.
‡Associate Professor, TU Delft Aerospace Engineering, Control and Simulation. Delft, The Netherlands. AIAA Member Grade.

Copyright © 2015 by Delft University of Technology. Published by the American Institute of Aeronautics and Astronautics, Inc., with permission.

quadrotor. Furthermore, it will demonstrate how this technology is becoming more accessible by using an inexpensive platform and an open source autopilot. This accessibility will hopefully result in a faster and wider exchange of resources.

When trying to achieve autonomous UAV flight, there are many levels of control to consider. In lower level control, the vehicle must be able to adjust its control surfaces or thrusters for rate or angle control. In the case of a quadrotor, it is simply to adjust the power to each individual engine for the desired attitude and accelerations. There have been many advances in lower level MAV control in recent years, both with modeled controllers, learning control and a mixture of the two.[5,8,10,11] Reinforcement learning has been implemented on a real quadrotor platform and compared to non-linear control methods for accurate hover[10] and improved vertical control[5] during non-linear disturbances. Other types of learning have been used to fine tune acrobatic maneuvers of quadrotors[7] within a well equipped flight arena[6] that provides information from a camera system. Expanding to other types of MAVs, a small helicopter was successfully controlled with RL in more difficult maneuvers by using a trained human pilot during the learning phase.[8] Finally, a quadrotor has used RL to hover and track a defined trajectory.[11]

For higher level control, an autonomous vehicle must make intelligent decisions in order to achieve its objective. High level control using reinforcement learning for real life ground robots has been explored to learn good behaviors of an E-Pet.[14] Finding an optimized path is of utmost importance when the vehicle is limited in battery life or mission time. Vehicles can also operate in an unknown environment where there is not enough information available to calculate the ideal path to their objective. This is a case where reinforcement learning can be effectively implemented.

Quadrotors are relatively easy to model for non-aggressive maneuvers and therefore can have fairly accurate lower level control with a PID controller in this flight regime. Reinforcement Learning can be used effectively in the high level control decisions to result in an optimized path for the UAV despite limited knowledge of the environment. The goal of the research presented in this paper is to demonstrate an application of a real life autonomous quadrotor, performing tasks determined by a high level decision making reinforcement learner. The second goal is to aim for further research with greater autonomy and capability to adapt to unforeseen environmental and vehicle changes.

The experiment presented is set up to symbolize just one of the many possible real world application that could benefit from a learning or adaptive controller. The quadrotor task is analogized to a reconnaissance task in a disaster scenario where MAVs are taking pictures of collapsed or damaged buildings and bringing the images back to headquarters. In order to complete this time-sensitive task efficiently, the quadrotor will use reinforcement learning to find the best path.

This paper presents a simulation of the quadrotor RL decision process in an unknown gridworld and the application to a real-life quadrotor in the state of the art tracking environment, the TU Delft Cyber Zoo [a]. The Cyber Zoo is a space outfitted with the OptiTrack [b], optical tracking system where ground robots and aerial vehicles can conduct their experiments using one or more agents. Lower level control of the quadrotor is supplied by Paparazzi,[12] an open source autonomous control software that provides a flexible autopilot system for fixed wing and multi-copter vehicles.

A background of Temporal Difference reinforcement learning as it is applied to this problem is outlined in Section II. The simulation setup and results will first be presented in Section III before the application setup and results in Section IV. An overview of all the results and conclusions is discussed in Section V.

II. Reinforcement Learning Preliminaries

A. Value Function

In Temporal Difference reinforcement learning, a Markov property is assumed. That is, the agent doesn't need to know any of its past actions to make its next action. In this way, the agent can use 2 pieces of information to make its decision:

1. Its current state
2. The value function (V-function)

[a] TU Delft Robotics Institute. http://robotics.tudelft.nl
[b] Optitrack™. http://www.naturalpoint.com

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

The value function, $V^{\pi}(s)$, represents the value that is assigned to each state, 's'. In this case, there are 216 states and so the value function will consist of 216 associated values. The agent wants to visit states with higher values. With each move the agent makes, it experiences part of the environment and can adjust the value function to learn what states are more desirable to visit.

The value function is updated by the Bellman Equation, Eq. 1. Where E_{π} is the expected reward in a state given a policy, π. Equation 2 shows the update rule used in this project where the reward function, r, and the step size, α are explained in further detail in Section III. Each state that is visited will be updated based on the rewards that it receives at that state, and the value of the next state it visits. This recursive property makes it possible to propagate a reward from one state to the states that surround it, and therefore make a transparent path of increasingly higher values to the reward states.

$$V^{\pi}(s) = E_{\pi}\{R_t|s_t = s\} = E_{\pi}\left\{\sum_{k=0}^{\infty}\gamma^k r_{t+k+1}|s_t = s\right\} \tag{1}$$

$$V(s) \leftarrow V(s) + \alpha(s)(r + \gamma V(s') - V(s)) \tag{2}$$

B. Action Policy

In reinforcement learning, the policy (π) decides what actions the agent will take. Introduced below are the policies that should be understood for this paper.

Random Every action has an equal probability of being selected.

Greedy The action that leads to the highest valued state will be chosen every time.

ϵ-greedy Takes the greedy action, ϵ percent of the time. Otherwise, a random action is taken (where the greedy action can still possibly be selected at random). We can calculate the probability that the greedy action,P^a_{greedy}, will be selected (Equation 3), or any other action, P^a_{other}, will be taken (Equation 4), where N_a is the number of possible actions.

$$P^a_{greedy} = \epsilon + \frac{1-\epsilon}{N_a} \tag{3}$$

$$P^a_{other} = \frac{1-\epsilon}{N_a} \tag{4}$$

After an infinite number of iterations, the V-function of the policy, V^{π}, will converge to V^{π^*}. The difference between different policies is due to the action probabilities associated to the policy. Note that in an ϵ-greedy policy, when $\epsilon = 0$ it is just a random policy and $\epsilon = 1$ is a Greedy policy. Since a higher valued action will be chosen more often with higher ϵ, the boundaries will be randomly hit less, the rewards will be found more frequently, and consequently the values in the fully converged V^{π^*} will be higher. Greedier policies will take much longer to converge due to lack of exploration in unvisited states. However, as long as there is an element of randomness, even a fully greedy policy will converge given enough iterations.

A full analysis of how the policy effects value function convergence can be found in the Results section.

III. Simulation

This section will describe how a reinforcement learning problem can be set up to imitate a reconnaissance quadrotor collecting pictures.

There are many real world applications that this kind of reinforcement learning concept could be used for. This experiment is set up to demonstrate a quadrotor with a mission to search for and collect images of collapsed buildings (reward states) in a disaster scenario. The agent must go to as many collapsed buildings as possible and take pictures. For each building it documents, some memory is used of its total storage space. When the memory is full, it must return to headquarters to upload the files. Then it starts over. Since the status of buildings in disasters are always changing, the quadrotor will need to prioritize finding

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

new collapsed buildings with getting updates from the known ones. As the quadrotor explores the area, the reinforcement learner will naturally balance its exploration of previously non-collapsed buildings but will more likely visit the states where it knows there are collapsed buildings to photograph.

A. Problem Setup

Using a reinforcement learning decision process, a quadrotor agent is simulated within a grid world of 3 collapsed buildings and an upload station. The agent gets rewards for visiting disaster sites until its figurative memory card is full of images. The more images the agent has collected, the greater the reward will be for visiting the upload station. The agent is set in a grid world that satisfies the Markov property.

The quadrotor agent's physical world consists of a 6x6 grid with the upload station located at [x y] = [6 6] and the disaster sites located at [1 3; 4 5; 6 1]. The locations of the disaster sites and upload station are visualized in Fig. 1. The boundaries of the grid world are not accessible.

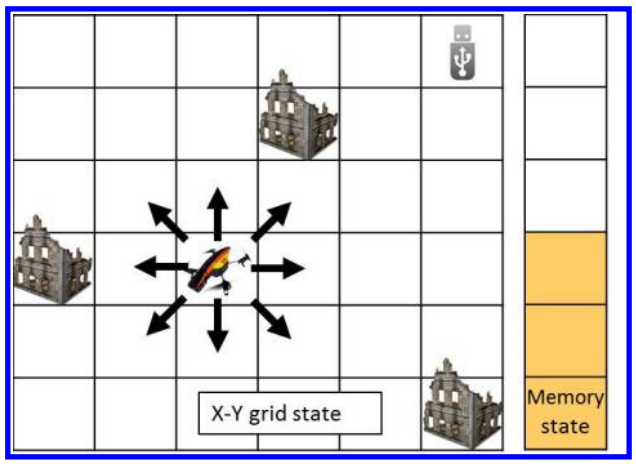

Figure 1. Disaster site reconnaissance RL concept: This figure demonstrates a discrete 6x6x6 grid world where the agent's location state is described by its x-y location in the grid and its memory state is represented by the bar on the right. The agent changes its memory state by visiting buildings or the upload station (symbolized by the USB stick) and can change location state by moving about the grid to adjacent squares(ie. 8 possible actions).

1. States, Actions, and Reward states

The state of the agent is 3 dimensional but is defined by 2 state values: Location state and memory state.

The memory state is an integer between 1 - 6 that represents how much memory the quadrotor is carrying on his body. This state is visually represented with the memory state bar along side the x-y grid in Fig. 1. The memory state initially starts at 1, meaning the quadrotor has collected no images. With each disaster site visited, the memory state value increases by 1 until the quadrotor is at maximum memory capacity at memory state = 6.

The current setup leads to a 6x6x6 three-dimensional (or a 36x6 two-dimensional), matrix of states with a total of 216 states. The grid size and the memory value maximum were arbitrarily chosen and could be adjusted as desired. However, as total number of states increases, so does the computational expense.

The agent can move to any adjacent square, including diagonal. Fig. 1 shows the grid world set up as well as the possible actions. If the quadrotor hits a boundary limit with the action, the quadrotor will stay in the same location state as the previous state.

The reward states are the disaster sites and the upload station. The agent gets penalized with a negative reward value for hitting the boundaries.

Regarding the reward values, a balance should be struck between the rewards for hitting the boundary, visiting a disaster site, and visiting the upload station. Initial values were selected from an experienced guess and then tuned by trial and error for better performance. A normal movement produces 0 reward.

Going out of bounds gives a '-1' penalty. The disaster site's reward is a constant of 8 at all states until memory state 6.

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

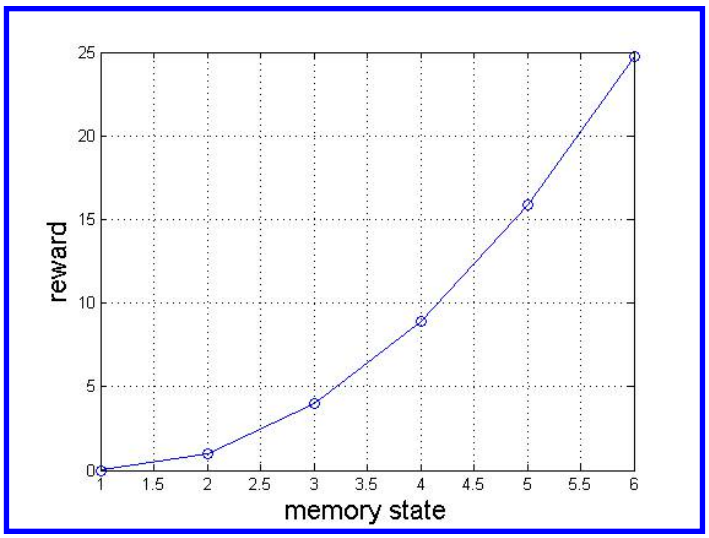

Figure 2. The reward at the upload station location state is a second order function of the memory state

Below is a description of the special states, their associated rewards, and the consequent automatic actions:

Upload station The agent receives a reward defined by the function in Fig. 2, is randomly relocated on the grid, and returned to memory state 1. Note: When the quadrotor's memory is full of images, there is no point to visit more disaster sites so there is no reward. Therefore, visiting the upload station should be overwhelmingly more rewarding at memory state 6 than any of the previous memory states. Furthermore, the upload station should progressively be more rewarding as the quadrotor has more images to bring back.

Disaster site If the agent is in memory state 1-5, it receives a reward of 8, is randomly relocated on the grid, and increases memory state +1. Random relocation is necessary, otherwise the agent would continually go the same disaster site if in a greedy policy. If the memory is full, then the reward is 0 (zero) and the next action is chosen by the current policy.

Out of bounds The agent is penalized by -1 and stays in the same state.

2. Parameter selection

This simulation required tuning of the reward values as already described in the previous section, and the Bellman equation update parameters as explained presently.

The discount factor, γ, is set to 0.9 to allow for quick propagation of the reward values through the states. The greater the value, the more future rewards will effect the current states value function update. This value was not varied for the simulation. The $\alpha(s)$ parameter in the update equation, Eq. 2, is used to create an average value at each state. It decreases for a state each time it is visited so that $\alpha(s) = 1/N_s$, where N_s is the number of times that state has been visited.

3. Policy selection

When the agent is first introduced to the environment, it will have no knowledge of the surroundings so it must first explore. Once it has identified the locations of some of the reward states it can begin to exploit that knowledge. Therefore, a scheduled ϵ-greedy policy has been selected for this problem where the agent begins with a purely random action policy. After some exploration, an ϵ-greedy policy is implemented with ϵ becoming greater and greater. Fig. 3 shows how ϵ was scheduled based on the error of the V-function (ΔV is calculated as the summed difference of all V-values after 100 iterations). ϵ was slowly increased and each time the policy was changed, the learning step size, α, was reset to 1 so that for example, an $\epsilon = 0.8$ policy was starting with an initial V-function that resulted from an $\epsilon = 0.6$ policy. This type of policy was

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

chosen for the practical reason that although a fully greedy policy is the optimal policy, during the learning phase it prevents certain states from being explored, which often results in slow convergence. The scheduled approach is a good fit for the disaster site example in that it allows for more exploration in the beginning when there could possibly be more collapsing buildings. As time goes by, it will be less likely that there will be newly collapsed buildings and so a more greedy policy will result in more frequent picture updates of the known sites.

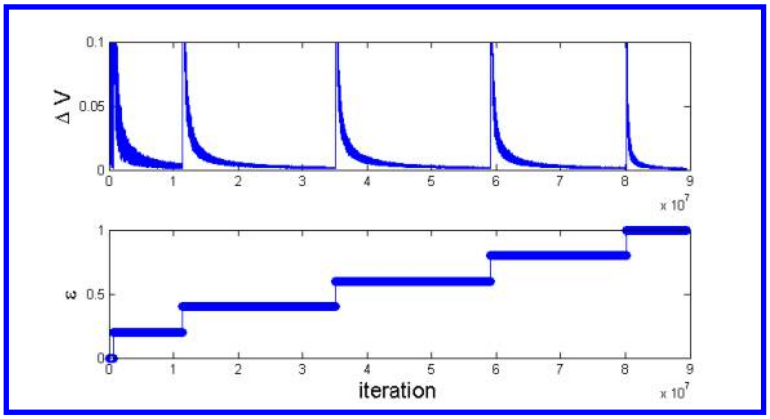

Figure 3. This scheduled ϵ-greedy policy increases ϵ as errors converge. The α value within the update equation (Eq. 2)is then reset, resulting in temporarily larger learning step sizes and therefore higher ΔV (errors)

B. Simulation Results

The results of this simulation will now be presented and briefly discussed.

For each policy, π, in the scheduled ϵ-greedy policy reinforcement learning training, the value function converges toward V^{π}. This was demonstrated in Fig. 3. Each intermediate policy was stopped short of converging after it met an error condition of $1e^{-3}$. The final policy was stopped when an error condition of $5e^{-4}$ was satisfied. This condition was chosen as a balance between accuracy and evaluation time. Therefore, the value function converged to a near optimal V^{π^*}.

The final value function and policy map is plotted in Fig. 4. The grid world is shown for memory states 1, 5, and 6. In memory state 1, the quadrotor will get no reward for going to the upload station so its greedy action is to go toward the nearest disaster site. In memory state 6, the memory is full and therefore the agent will only get a reward to go to the upload station. Every memory state in between shows a blend of the two. Memory state 5 is important in that it should still go to take pictures at a disaster sites, but if the agent is nearby the upload station it is also a good action to go there.

Using the policy map with the arrows, it is easy to see from any location where the agent will go. Just follow the arrows. In some cases, it seems like an indirect path is taken. However, it must be considered that each square in the grid is one step. To move diagonally is indeed physically more distance, but the problem setup is not considering this as a longer path because the algorithm only considers number of states to the reward. In reality, the chosen path as shown by the arrows is sometimes not the only optimal path. The arrows only show one greedy direction, but in reality there are some sates where more than one action results in a very similar next value. If the value function were fully converged the values of the two or three contending states would be exactly the same. However, full convergence requires iterations $t \to \infty$, and will therefore always have some error. It is worth while to note that full value function convergence is not necessary to have the optimized policy. It is true that it takes much less iterations to find one of the optimal policies.

IV. Flight tests

This section explains the setup and results of the flight tests with a real quadrotor performing the reinforcement learning task described in Section III. Firstly, the resources used in the experiment will briefly be explained. Secondly, the setup of the experiment will detail how all these components interact together

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

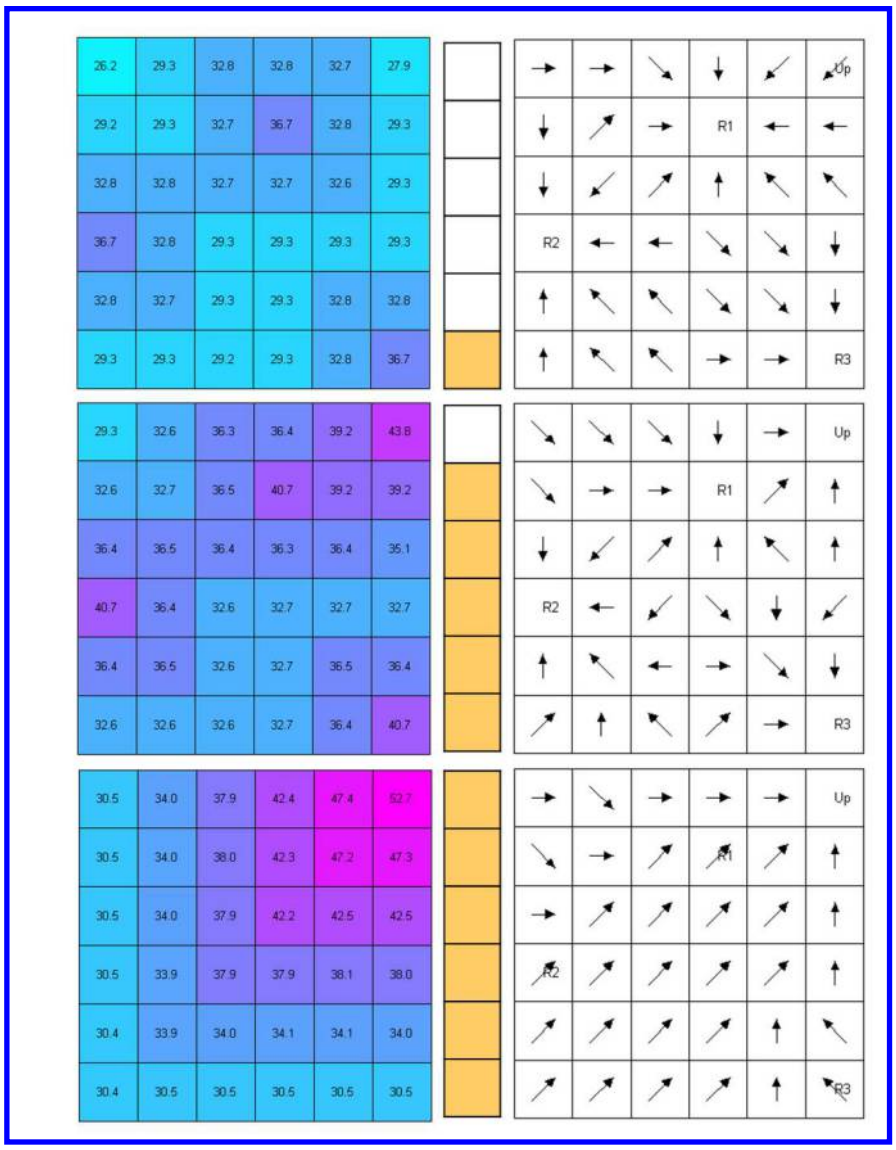

Figure 4. V^{π} **and policy map at memory states 1,5, and 6 for a scheduled policy that converges to a near optimal** V^{*}**.**

to recreate a real life version of the simulation. Lastly, the results from the flight tests will be presented and briefly discussed.

A. Resources

Formerly, to work with a quadrotor platform, researchers often used in-house quadrotor designs which were often time intensive to model and tune.[2,4] Since designing and building a quadrotor is not the main research intention for this experiment, it is very welcome to have a commercial quadrotor available which makes this research not only more convenient for this project but also more accessible to other researchers in the field. This benefit is the same for the other resources discussed such as open source software. The uses and benefits of these resources in the context of this research will now be discussed.

1. AR-Drone 2 quadrotor

A quadrotor is an unmanned aerial vehicle consisting of 4 engines that spin independently of each other. Adjusting the thrust produced by each engine gives the necessary control to perform general motion as well

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

as other aggressive maneuvers.[7] The Parrot® AR-Drone 2 [c] as shown in Fig. 5, is a commercially available quadrotor intended for recreational use. Users can download a free app onto a smartphone or tablet and manually pilot the vehicle from there while receiving live streaming on their screen from the camera on the quadrotor. The AR-Drone 2 is also promising for research and educational purposes[3] as it is relatively inexpensive, and robust to damage. Most importantly for this project is that its built-in software can be overwritten.

Figure 5. The AR-drone 2 is a relatively inexpensive quadrotor MAV that can be purchased for recreational uses or for research. Picture credits by Parrot.com

Some useful features of the AR-Drone quadrotor for this research are:

- Robust structural design of carbon fiber tubes and insulating foam can withstand many crashes
- Linux 2.6.32 (with ability to run Paparazzi software)
- Wi-Fi b g n
- GPS
- Precision control (About ±5cm in a tracked hover)

2. *TU Delft Cyber Zoo*

The TU Delft Cyber Zoo [d] was officially opened in March 2014 as a research and test laboratory for ground robots and aerial vehicles. Its space consists of a floor area that is 10 x 10 meters and extends 7 meters high. The space is contained inside an aluminum truss frame which holds nets to ensure the safety of people and surrounding equipment. Also supported on the truss structure are 24 cameras. These cameras are part of the *Optitrack* [e]*: Motive Tracker* optical tracking system. The quadrotor "wears" a special hull with reflective balls so that the cameras can track it. Within the Cyber Zoo boundaries, up to 32 rigid bodies can be tracked with high precision accuracy. The system can also be used as a simulated GPS. Therefore, as long as an experiment is not using any feedback from the Cyber Zoo, everything done indoors can be translated to an outdoor environment.

This system is not the first of its kind being used for quadrotors.[6] ETH Zurich has shown with their aerobatic quadrocopters how powerful a designated space with tracking can be. This flight test will use the tracking system only for simulated GPS and for evaluating performance. Further capabilities including the high precision information from the cameras for feedback are available but not currently used. Therefore, this experiment is easy to move to an outdoor environment as long as GPS is available. The complete system as it is used is illustrated in Fig. 6. The computer running the OptiTrack software has a wired connection to the laptop running Paparazzi and from there can broadcast the GPS signal via WiFi to the quadrotor.

3. *Paparazzi Open Source Autopilot*

Paparazzi [f] is a fully open source free autopilot for fixed wing and multi-copter UAVs. The development is particularly active within ENAC University in France, Delft University of Technology, and Utah State

[c]Parrot AR-Drone 2. http://ardrone2.parrot.com/
[d]TU Delft Robotics Institute. http://robotics.tudelft.nl
[e]Optitrack™. http://www.naturalpoint.com
[f]Paparazzi Free Autopilot. http://wiki.paparazziuav.org

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

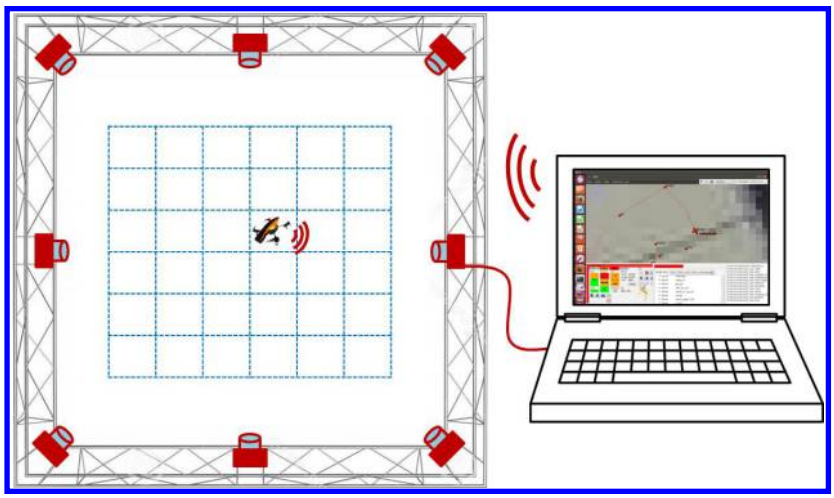

Figure 6. Cyber Zoo experimental setup: 24 cameras track the quadrotor. From this system, the quadrotor receives a simulated GPS signal. That position is transmitted via WiFi between a laptop and the quadrotor. Based on the location state and the memory state, a waypoint command will be sent to the quadrotor for it to execute.

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

University.

The quadrotor capabilities in Paparazzi include a model of a standard quadrotor that is close enough to the AR-Drone to function as a good inner loop controller given good tuning and standard non-aggressive maneuvers. Simulation of a flight plan in an intuitive GUI is available in addition to use as a Ground Control Station (GCS) for real flights. An example of the Paparazzi GCS interface can be seen in Fig. 7(*left*).

In the indoor setting, Paparazzi connects via WiFi to the quadrotor to give commands and via a wired connection to the Cyber Zoo computer to receive the position of the vehicle. In an outdoor situation, GPS would be picked up by the AR-Drone and used there directly.

Paparazzi is a modular platform and therefore has made it easy to add functionality by means of custom modules. Initializations, periodic and event functions can be added without effecting the main autopilot software. This makes customization very flexible and safe.

The high level position control of the vehicle is controlled by the flight plan. Waypoints and commands involving these waypoints are written into the flight plan. The autopilot software and the flight plan is compiled and uploaded onto the quadrotor computer. Once uploaded, the names and numbers of the waypoints and types of commands cannot be changed; however, the location of the waypoints can be modified at any time. Therefore, it is important to plan the method for which the waypoints will be used. In this experiment, the method of use is shown in Fig. 7(*right*). A new module was created to move the waypoints based on reinforcement learning logic.

B. System Setup

Now that the components of the whole system are understood, the next step is to discuss how they work together to make the experiment function. The controller consists of the Paparazzi autopilot with a reinforcement learning module. Once the software is uploaded to the quadrotor computer, the user can send commands from the GCS flight plan via WiFi to the quadrotor. By calling the RL module function from the flight plan, the RL algorithm in the module chooses the next state based on the current position. The next state is translated to a waypoint position for Paparazzi to understand. If the quadrotor is currently at waypoint 1 (wp1), then wp2 will move to the next position. This concept of moving waypoints and then going to it is shown in Fig. 7(*right*). Paparazzi's inner loop controller, which is already uploaded onboard, will translate the waypoint command to specific low level control commands.

Input and output to the quadrotor computer is possible by use of a usb flash drive that must be plugged into the AR-drone connector before it is powered on.

Figure 7. ***(left)*** **Paparazzi GCS (Ground Control Station) is the main interface for the user. In the GUI it shows a 2D map with a flight path (cyan line). The quadrotor status is shown in strips box (bottom-left) along with some buttons for common commands. Commands such as "go p1" or "stay p2", can be seen in the Flight Plan tab in the Notbook box (bottom-middle). Commands that have been made are printed in the console box (bottom-right).**
(right) **The quadrotor is commanded by paparazzi to go to a waypoint. Waypoints cannot be added online but they can be moved. Therefore, after each action decision, the waypoint will be moved to the desired state. Commands alternate between going to waypoint 1 and waypoint 2 from the other.**

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

C. Results

Two flight tests will now be explained and the results presented. Due to battery limitations, the quadrotor is not able to perform millions of iterations like is possible in simulation. Therefore, the flight test was split into two separate tests that would demonstrate the important reinforcement learning concepts in a smaller number of iterations.

1. Test 1: Exploration and Learning

The first flight test demonstrates the exploration phase of reinforcement learning. Within one test flight, about 200 iterations can be performed in 10 minutes before the battery depletes. To better visualize the results, this test was limited to a 6x6 grid of 36 states instead of the normal 6x6x6 grid of 216 states. The memory state was kept at 1, where there is no reward for visiting the upload station.

The result was that the vehicle successfully moved to its randomly initialized spot on the grid within the Cyber Zoo and continued to randomly choose actions and execute them. The resulting value function after 200 iterations from that flight is shown in Fig. 8.

The result from this test flight shows that each of the disaster sites were visited at least once, yielding a reward at that state. The reward was also able to propagate to some of the neighboring states. The boundary was hit several times as can be seen by the negative values in border states. Some of the states were never visited as can be seen by a zero value.

Looking at the greedy policy map in Fig. 8, we see that even though the value function is not even close to being converged, one of the optimal policies is already found. This means, that if a greedy policy were implemented at this point, the vehicle would always find a direct path to the nearest reward spot.

To validate the result against the Matlab simulation, all randomly generated numbers were saved to a file and used in the simulation to yield the same value function.

2. Test 2: Executing a greedy policy from an imported value function

The second flight test demonstrates the ability to upload an existing value function and follow a non-random policy determined by that value function. In this case, the quadrotor loaded a nearly-converged value function from 5e7 iterations of ϵ-greedy simulation. The flight followed a fully greedy policy. The result is shown in Fig. 9. There is nothing surprising with these results. It follows the greedy path as was desired.

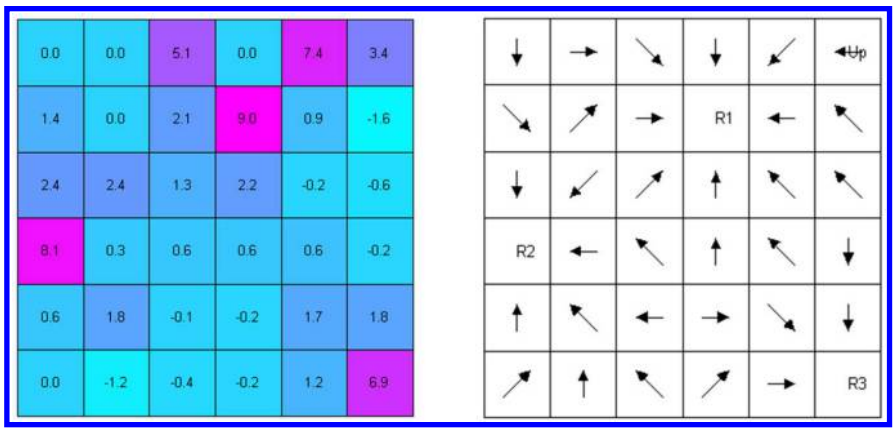

Figure 8. V^{π} **and policy map at memory states 1 for a random policy after 200 iterations in actual flight. After 200 iterations, one of the optimal solutions for a greedy policy has already been found.**

Fig. 9 shows the grid world at each of the memory states. In memory state 1, it is randomly relocated in state [3 1] and it takes the most direct path to the nearest reward state. Each time it reaches a reward state it randomly relocates onto the grid in the next memory state. The random relocation spot is shown by a solid red circle. The path it takes is shown with black arrows, and the final reward spot it navigates to is a dashed circle. Following the circles and arrows shows us the expected greedy behavior.

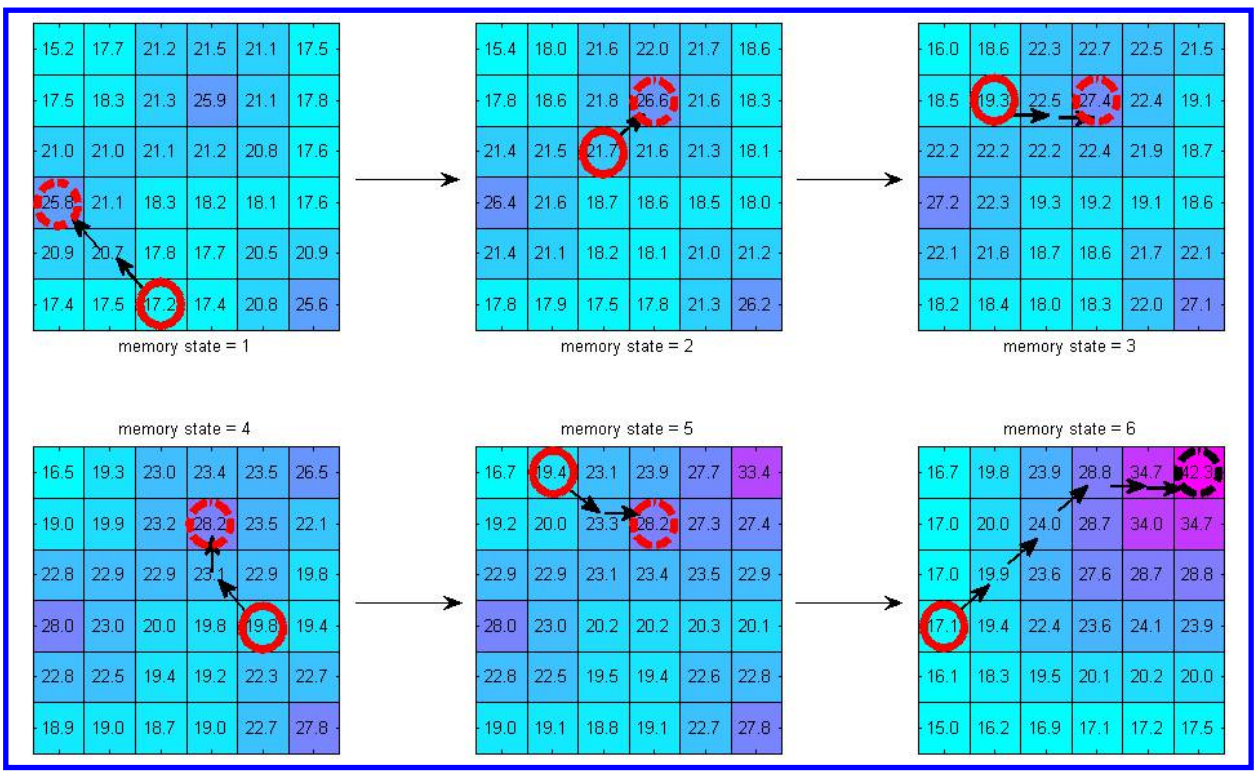

Figure 9. The imported value function at each memory state and the actions chosen by the vehicle with a greedy policy. The solid red circle shows the location in that memory state that the vehicle was randomly generated in. The arrows show the path the greedy policy the vehicle followed, and the dashed red circle is the reward sport it found before beginning at a random location in the next memory state.

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

V. Conclusion

This paper has presented a reinforcement learning task for a quadrotor in an unknown environment. By learning from interactions with the environment, this learning approach works towards more adaptive and robust control laws for autonomous MAVs.

Firstly, a simulation grid world was setup where the agent learned a near optimal value function using a scheduled ϵ-greedy to full greedy policy. In the end of the simulation, the agent chooses an optimal mission path through the grid world. Secondly, a real life quadrotor experiment was presented as an application of the simulation. The results show that the quadrotor can learn and implement a policy as well as the simulation but is limited in the number of iterations it can perform. However, since the optimal greedy policy is converged long before the value function converges it is therefore not necessary to converge the value function over millions of iterations to such a small error. This can make learning in-flight more feasible.

The flight tests show a hybrid approach where learning a value function is initiated in flight, converged over many iterations in simulation, and then is used again in a flight test. This is one approach to solve the conflict between limited battery life and the need for many iterations.

Not only does this paper present a successful implementation of a reinforcement learning task, it also is a demonstration of an inexpensive and accessible means of research by using a commercially available quadrotor and an open source autopilot. Added accessibility will promote collaboration and accountability in this field, which will hopefully result in faster progress.

This work is just one step toward more autonomous quadrotor flight. Further research will look into the challenges of working in the continuous domain, onboard reward or state recognition, adaptive learning for changing environments, and use of learning algorithms for improvement of inner loop control for multiple platforms.

References

[1]Sutton, R. and Barto, A., *Introduction to Reinforcement Learning*. MIT Press, Cambridge, MA, USA, 1998.

[2]Raza, S.A., and Gueaieb, W. " Intelligent Flight Control of an Autonomous Quadrotor", *Motion Control*, Federico Casolo (Ed.), InTech, 2010, Chap 12.

[3]Krajník, T., Vonásek, V., Fiŝer, D., and Faigl, J. "AR-Drone as a Platform for Robotic Research and Education", *Research and Education in Robotics - EUROBOT 2011*. Communications in Computer and Information Science. Vol 161, 2011, pp 172 − 186.

[4]Hoffmann, G.M., Rajnarayan,D.G., Dostal, D., Jang, J.S., Waslander, S.L., and Tomlin, C.J. "The Stanford testbed of autonomous rotorcraft for multi agent control(STARMAC)", *23rd Digital Avionics Systems Conference*, Salt Lake City, UT, 2004.

[5]Waslander, S.L., Hoffmann, G.M., Jang, J.S., and Tomlin, C.J. "Multi-Agent Quadrotor testbed control design: Integral Sliding Mode vs. Reinforcement Learning", *IEEE International Conference on Intelligent Robots and Systems*, 2005, pp 3712 − 3717.

[6]Lupashin, S., Hehn, M., Mueller, M. W., Schoellig, A. P., Sherback, M., and D'Andrea, R. "A platform for aerial robotics research and demonstration: The Flying Machine Arena". *Mechatronics*, 24(1), 2013, pp 41 − 54.

[7]Purwin, O., and DAndrea, R. "Performing aggressive maneuvers using iterative learning control". *IEEE International Conference on Robotics and Automation*. 2009, pp 1731-1736.

[8]Abbeel, P., Coates, A., Quigley, M., and Ng, A., "An Application of Reinforcement Learning to Aerobatic Helicopter Flight". *Advances in Neural Information Processing Systems 19*. MIT Press, 2007.

[9]Ng, A.Y., Kim, H.J., Jordan, M., and Sastry, S. "Autonomous helicopter flight via reinforcement learning". *Neural Information Processing Systems 16*. 2004.

[10]Bou-Ammar, H., Voos, H., and Ertel, W. "Controller design for quadrotor UAVs using reinforcement learning". *IEEE International Conference on Control Applications*. Yokohama, Japan, 2010. pp 2130 − 2135.

[11]Santos, SRBd., Givigi, SN Jr., and Jnior, CLN. "An Experimental Validation of Reinforcement Learning Applied to the Position Control of UAVs". *IEEE International Conference on Systems, Man, and Cybernetics*. Seoul, Korea, 2012. pp 2796-2802.

[12]Remes, B., Hensen, D., Tienen, F. Van, Wagter, C. De, Horst, E. Van Der, and Croon, G. De. "Paparazzi : how to make a swarm of Parrot AR Drones fly autonomously based on GPS". *International Micro Air Vehicle Conference and Flight Competition*. Toulouse, Fr , 2013. pp 17 − 20.

[13]Zamstein, L. M., Arroyo, A. A., Schwartz, E. M., Keen, S., and Sutton, B. C. "Koolio : Path planning using reinforcement learning on a real robot platform". *Florida Conference on Recent Advances in Robotics*. Miami, Florida, 2006.

[14]Hsu, C.-W., and Liu, A. "High-level behavior control of an e-pet with reinforcement learning". *IEEE International Conference on Systems, Man and Cybernetics*. Istanbul, Turkey, 2010. pp 29 − 34.

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990

This article has been cited by:

1. Stephen Verbist, Tommaso Mannucci, Erik-Jan Van Kampen. The Actor-Judge Method: safe state exploration for Hierarchical Reinforcement Learning Controllers . [Citation] [PDF] [PDF Plus]
2. Jaime Junell, Tommaso Mannucci, Ye Zhou, Erik-Jan Van Kampen. Self-tuning Gains of a Quadrotor using a Simple Model for Policy Gradient Reinforcement Learning . [Citation] [PDF] [PDF Plus]

Downloaded by TU DELFT on October 10, 2018 | http://arc.aiaa.org | DOI: 10.2514/6.2015-1990